beast网络库搭建http服务器

Posted on 2023-06-27 09:47:33 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2RlhDCg4eedYme46C6ddo4cKcFN)

简介

前面的几篇文章已经介绍了如何使用asio搭建高并发的tcp服务器，以及http服务器。但是纯手写http服务器太麻烦了，有网络库beast已经帮我们实现了。这一期讲讲如何使用beast实现一个http服务器。

连接类

我们先实现http_server函数

```
1. void http_server(tcp::acceptor& acceptor, tcp::socket& socket)
2. {
3.     acceptor.async_accept(socket,
4.         [&](beast::error_code ec)
5.         {
6.             if (!ec)
7.                 std::make_shared<http_connection>(std::move(socket))->start();
8.             http_server(acceptor, socket);
9.         });
10. }
```

http_server中添加了异步接收连接的逻辑，当有新的连接到来时创建 `http_connection` ,然后启动服务，新连接监听对端数据。接下来http_server继续监听对端的新连接。
连接类 `http_connection` 里实现了start函数监听对端数据

```
1. void start()
2. {
3.     read_request();
4.     check_deadline();
5. }
```

处理读请求,将读到的数据存储再成员变量 `request_` 中，然后调用 `process_request` 处理请求

```
void read_request()
{
    auto self = shared_from_this();

    http::async_read(
        socket_,
        buffer_,
        request_,
        [self](beast::error_code ec,
               std::size_t bytes_transferred)
        {
            boost::ignore_unused(bytes_transferred);
            if (!ec)
                self->process_request();
        });
}
```

`check_deadline` 主要时用来检测超时，当超过一定时间后自动关闭连接，因为http请求时短链接

```
    void
        check_deadline()
    {
        auto self = shared_from_this();

        deadline_.async_wait(
            [self](beast::error_code ec)
            {
                if (!ec)
                {
                    // Close socket to cancel any outstanding operation.
                    self->socket_.close(ec);
                }
            });
    }
```

`process_request` 函数中区分请求的类型，进行不同类型的处理如post还是get请求

```
void process_request()
{
    response_.version(request_.version());
    response_.keep_alive(false);

    switch (request_.method())
    {
        case http::verb::get:
            response_.result(http::status::ok);
            response_.set(http::field::server, "Beast");
            create_response();
            break;
        case http::verb::post:
            response_.result(http::status::ok);
            response_.set(http::field::server, "Beast");
            create_post_response();
            break;
        default:
            // We return responses indicating an error if
            // we do not recognize the request method.
            response_.result(http::status::bad_request);
            response_.set(http::field::content_type, "text/plain");
            beast::ostream(response_.body())
                << "Invalid request-method '"
                << std::string(request_.method_string())
                << "'";
            break;
        }

    write_response();
}
```

`create_response` 函数中解析了不同的路由处理get请求

```
void
    create_response()
{
    if (request_.target() == "/count")
    {
        response_.set(http::field::content_type, "text/html");
        beast::ostream(response_.body())
            << "<html>\n"
            << "<head><title>Request count</title></head>\n"
            << "<body>\n"
            << "<h1>Request count</h1>\n"
            << "<p>There have been "
            << my_program_state::request_count()
            << " requests so far.</p>\n"
            << "</body>\n"
            << "</html>\n";
    }
    else if (request_.target() == "/time")
    {
        response_.set(http::field::content_type, "text/html");
        beast::ostream(response_.body())
            << "<html>\n"
            << "<head><title>Current time</title></head>\n"
            << "<body>\n"
            << "<h1>Current time</h1>\n"
            << "<p>The current time is "
            << my_program_state::now()
            << " seconds since the epoch.</p>\n"
            << "</body>\n"
            << "</html>\n";
    }
    else
    {
        response_.result(http::status::not_found);
        response_.set(http::field::content_type, "text/plain");
        beast::ostream(response_.body()) << "File not found\r\n";
    }
}
```

`create_post_response` 处理了post请求中的一部分路由

```
void create_post_response() {
    if (request_.target() == "/email")
    {
        auto& body = this->request_.body();
        auto body_str = boost::beast::buffers_to_string(body.data());
        std::cout << "receive body is " << body_str << std::endl;
        this->response_.set(http::field::content_type, "text/json");
        Json::Value root;
        Json::Reader reader;
        Json::Value src_root;
        bool parse_success = reader.parse(body_str, src_root);
        if (!parse_success) {
            std::cout << "Failed to parse JSON data!" << std::endl;
            root["error"] = 1001;
            std::string jsonstr = root.toStyledString();
            beast::ostream(this->response_.body()) << jsonstr;
            return ;
        }

        auto email = src_root["email"].asString();
        std::cout << "email is " << email << std::endl;

        root["error"] = 0;
        root["email"] = src_root["email"];
        root["msg"] = "recevie email post success";
        std::string jsonstr = root.toStyledString();
        beast::ostream(this->response_.body()) << jsonstr;
    }
    else
    {
        response_.result(http::status::not_found);
        response_.set(http::field::content_type, "text/plain");
        beast::ostream(response_.body()) << "File not found\r\n";
    }
}
```

write_response 发送请求

```
void write_response()
{
    auto self = shared_from_this();

    response_.content_length(response_.body().size());

    http::async_write(
        socket_,
        response_,
        [self](beast::error_code ec, std::size_t)
        {
            self->socket_.shutdown(tcp::socket::shutdown_send, ec);
            self->deadline_.cancel();
    });
}
```

总结

本文介绍了如何使用beast网络库实现http服务器

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2752) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(576) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)